Artisan Marketplace Platform

System Design & Architecture Document — Version 1 (MVP)

A backend-focused Java / Spring Boot learning project connecting clients seeking home renovation services with freelance artisans (plumbers, electricians, welders, painters, builders).

Prepared as a collaborative design exercise — Software Development Lifecycle, Phase 1 & 2

Contents

1. Problem Statement & Scope

Freelance artisans (plumbers, electricians, welders, painters, builders) are often difficult for prospective clients to discover, vet, and hire in a trustworthy way. This platform is a two-sided marketplace connecting clients who need home renovation work with artisans offering their services for a fee.

V1 is scoped as a single-region, layered monolith web application, prioritizing correctness of the core job lifecycle, trust signals (ratings, verified qualifications, portfolio evidence), and location-aware matching, over breadth of features.

2. Functional Requirements

Client Capabilities

- Register and log in; manage profile
- Browse and search artisans (by trade, location, rating)
- View an artisan's profile (skills, rate, reviews, portfolio, qualifications)
- Post a job publicly (“open job”) and receive ranked artisan matches
- Directly request a specific artisan found via search
- Review and accept an artisan's response to an open job
- Track job status through its lifecycle
- Pay for a completed job via Paystack
- Leave a rating and review after job completion

Artisan Capabilities

- Register and log in; build a profile (trade(s), bio, rate, location)
- Upload portfolio media (photos/videos) and qualifications as evidence of work
- Appear in client searches and ranked matches, filtered by trade and proximity
- Receive and respond to direct job requests (accept/decline)
- Browse open jobs near them and submit a response/proposal
- Update job progress through its lifecycle
- Receive payment once the client confirms completion
- View their own ratings and reviews

Shared / Platform Capabilities

- Authentication and role-based authorization (CLIENT, ARTISAN, ADMIN)
- In-app notifications for job status changes (email is a stretch goal, not v1)

3. Non-Functional Requirements

Security: Passwords hashed (BCrypt); role-based access control on every endpoint; payment endpoints additionally verified server-side against Paystack, never trusting client-supplied payment status.

Data Integrity: Job status transitions follow a strict, enforced state machine — a job cannot skip states (e.g. OPEN directly to COMPLETED).

Performance: Location-based artisan search must use a spatial index (PostGIS) rather than scanning every artisan row and computing distance in application code.

Maintainability: Layered architecture with clear separation of concerns; payment and matching logic isolated in the service layer, never embedded in controllers.

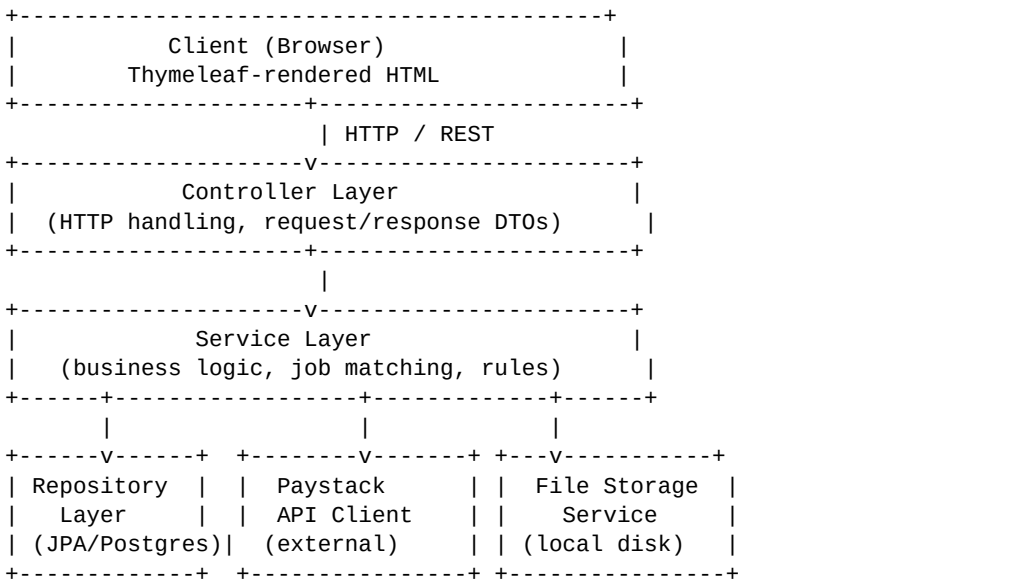
Resource constraints: Designed to run on modest development hardware (4GB RAM) — informs choices like Maven over Gradle and Thymeleaf over a separate Node-based frontend for v1.

4. Technology Stack

Layer	Choice	Rationale
Language / Runtime	Java 25	Current LTS-adjacent release; already installed
Framework	Spring Boot 3.x	Industry standard across SA Java job listings
Build tool	Maven	Lighter on RAM than Gradle; SA market standard
Persistence	Spring Data JPA + Hibernate	ORM layer over PostgreSQL
Database	PostgreSQL + PostGIS	PostGIS required for geospatial artisan matching
Auth	Spring Security + JWT	Stateless tokens; no server-side session storage
Payments	Paystack API	Strong South African / African support; sandbox mode
Media storage	Local disk (v1), abstracted via interface	Swappable for S3/Cloudinary later without touching business logic
Testing	JUnit 5 + Mockito	Written alongside each feature, not after
Frontend (v1)	Thymeleaf	Server-side rendering; avoids running a separate Node process on limite

5. System Architecture Overview

V1 is built as a layered monolith — not microservices — since a single well-structured application is the appropriate scope for a learning project of this size, and reflects how most South African junior Java roles operate day to day.



6. Project Structure (Package-by-Feature)

Code is organized by feature/domain rather than by technical layer. This groups everything that changes together in one place (high cohesion), rather than scattering one feature's controller, service, and repository across three separate top-level folders as “package-by-layer” would.

```
com.artisanmarketplace
|-- job/
|   |-- Job.java
|   |-- JobController.java
|   |-- JobService.java
|   |-- JobRepository.java
|   |-- JobStatus.java (enum)
|   `-- dto/
|-- artisan/
|   |-- ArtisanProfile.java
|   |-- ArtisanController.java
|   |-- ArtisanService.java
|   |-- ArtisanRepository.java
|   |-- Portfolio.java
|   `-- dto/
|-- client/
|   `-- ... (User/Client-facing endpoints)
|-- auth/
|   |-- JwtService.java
|   |-- SecurityConfig.java
|   `-- ...
```

```
|-- payment/
|   |-- PaymentGateway.java (interface)
|   |-- PaystackService.java (implementation)
|   `-- ...
`-- common/
    |-- exception/
    `-- config/
```

7. Entity-Relationship Design

Design decision: Client and Artisan are modeled via composition, not JPA inheritance. A single *User* table holds shared identity and auth data; a separate *ArtisanProfile* table (1-to-1 with User) holds artisan-specific fields only. This favors composition over inheritance and avoids unnecessary Hibernate inheritance-strategy complexity, since Client and Artisan share no behavior beyond login.

Entity	Key Fields	Relationship
User	id, email, password_hash, role, full_name, phone, created_at	Base identity + auth
ArtisanProfile	id, user_id (FK, unique), bio, hourly_rate, location, avg_rating, verified	1-to-1 with User
Trade	id, name (PLUMBER, ELECTRICIAN, ...)	Lookup table
ArtisanTrade	artisan_id (FK), trade_id (FK)	M:N join (artisan can have multiple trades)
PortfolioMedia	id, artisan_id (FK), media_url, media_type, caption	N:1 with ArtisanProfile
Qualification	id, artisan_id (FK), title, issuing_body, document_url, verified	N:1 with ArtisanProfile
Job	id, client_id (FK), assigned_artisan_id (FK, nullable), trade_required (FK), title, description, location, origin, status, budget, crea	
JobResponse	id, job_id (FK), artisan_id (FK), message, proposed_rate, status	Populated only when origin = OPEN_POST
Review	id, job_id (FK, unique), client_id (FK), artisan_id (FK), rating, comment	1-to-1 with completed Job
Payment	id, job_id (FK, unique), amount, status, paystack_reference	1-to-1 with Job

Relationship Diagram

```
User (1) ------------- (0..1) ArtisanProfile
                               |
                               |-- (1:N) PortfolioMedia
                               |-- (1:N) Qualification
                               `-- (M:N via ArtisanTrade) Trade

User [Client] (1) ------------- (N) Job
ArtisanProfile (1) ------------- (0..N) Job  [assigned_artisan_id]
Job (1) ------------- (0..N) JobResponse ------------- (N:1) ArtisanProfile
Job (1) ------------- (0..1) Review
Job (1) ------------- (0..1) Payment
```

Why JobResponse exists separately: for a direct request there is no “responding” — the artisan simply accepts or declines the job itself (a status change on Job). JobResponse only holds competing proposals when multiple artisans respond to the same open post, so the table is never populated with rows that don't conceptually apply.

Why location lives on both ArtisanProfile and Job: an artisan has a home base, while a job has a job-site — these are not always the same place, and the matching query must compare both.

8. Core Domain Flow (Happy Path)

1. Client posts a job → JobController → JobService.createJob() → saved via JobRepository, status = OPEN
2. Matching → JobService queries ArtisanRepository for artisans matching trade + within radius (PostGIS), ranked by rating and portfolio completeness
3. Client reviews matches and either accepts a suggested artisan or searches manually → status → REQUESTED, assigned_artisan_id set
4. Artisan accepts → status → ACCEPTED → IN_PROGRESS
5. Artisan marks job complete → status → COMPLETED
6. Client triggers payment → PaymentService calls Paystack API → on success, Payment record saved, linked to Job
7. Client leaves a review → Review entity created, linked to Job and ArtisanProfile, feeds back into future ranking

9. Key Architectural Decisions & Rationale

Single Job entity, not separate Direct/Open models

A direct request and an open post both resolve to the same end state — one job, eventually assigned to one artisan. Origin is metadata about how the job started, not a reason to fork the entity (DRY + Single Responsibility).

Composition over inheritance for User/Artisan

Client and Artisan share only login concerns, not behavior, so a base User table plus a separate ArtisanProfile avoids unnecessary JPA inheritance-mapping complexity.

Rule-based ranking, not a recommendation engine

Matching is filtering (trade + radius) plus weighted sorting (rating, portfolio completeness) — a solvable SQL/scoring problem, deliberately scoped away from machine-learning-based recommendations, which would be a different project entirely.

Stateless JWT authentication

No server-side session storage — keeps the app lightweight on constrained development hardware and simplifies any future horizontal scaling.

Service layer owns all business rules

Controllers stay thin, translating HTTP to service calls only. Job-matching logic, status transitions, and payment logic are testable independently of the web layer.

External services abstracted behind interfaces

PaymentService depends on a PaymentGateway interface, not directly on the Paystack SDK; file storage is similarly abstracted. This is the Dependency Inversion Principle in practice — swapping providers later never touches business logic.

PostgreSQL + PostGIS over a generic relational database

Location-based matching is a first-class v1 requirement, and PostGIS provides indexed spatial queries that would otherwise require inefficient application-level distance calculations.

10. Job Status State Machine

An additional state beyond the originally scoped lifecycle was introduced here: **AWAITING_CONFIRMATION**. Without it, an artisan could unilaterally mark a job complete, which is precisely the moment payment is triggered — a trust gap. Requiring client confirmation before COMPLETED protects the payment relationship.

Status	Meaning
OPEN	Job posted, no artisan assigned (open posts only)
REQUESTED	An artisan is attached, awaiting their response
ACCEPTED	Artisan has accepted, work not yet started
IN_PROGRESS	Artisan has started the work
AWAITING_CONFIRMATION	Artisan says it's done; waiting on client to confirm
COMPLETED	Client confirmed — job closed, eligible for payment
CANCELLED	Terminal — withdrawn/declined before work started

Transition Table

From	To	Triggered by	Notes
OPEN	REQUESTED	Client	Accepts a JobResponse, or direct-request job created straight into this state
OPEN	CANCELLED	Client	Client withdraws the post
REQUESTED	ACCEPTED	Artisan	Artisan accepts
REQUESTED	OPEN	Artisan	Declines and origin = OPEN_POST (other responses may be pending)
REQUESTED	CANCELLED	Artisan/Client	Declines and origin = DIRECT_SEARCH, or client withdraws first
ACCEPTED	IN_PROGRESS	Artisan	Work begins
ACCEPTED	CANCELLED	Client/Artisan	Mutual cancellation before work starts
IN_PROGRESS	AWAITING_CONFIRMATION	Artisan	Artisan marks work done
AWAITING_CONFIRMATION	COMPLETED	Client	Unlocks payment and review

Deliberately out of scope for v1: disputes (client rejects the “done” claim) and cancellation once IN_PROGRESS. Both are real needs for a mature platform but introduce refunds, partial payment, and arbitration — scope deliberately deferred and documented as known future work.

Implementation Approach: Guard Clause, not full State Pattern

Rather than building a full Gang-of-Four State pattern (a class per state implementing a shared interface), v1 uses a lightweight guard-clause approach: a JobStatus enum holds an explicit map of allowed transitions, and JobService consults it before ever writing a status change. This keeps the transition rule

as a Single Source of Truth without the overhead of full State classes, which would be premature here since per-state behavior is minimal.

```
public enum JobStatus {
    OPEN, REQUESTED, ACCEPTED, IN_PROGRESS,
    AWAITING_CONFIRMATION, COMPLETED, CANCELLED;

    private static final Map<JobStatus, Set<JobStatus>> ALLOWED_TRANSITIONS = Map.of(
        OPEN, Set.of(REQUESTED, CANCELLED),
        REQUESTED, Set.of(ACCEPTED, OPEN, CANCELLED),
        ACCEPTED, Set.of(IN_PROGRESS, CANCELLED),
        IN_PROGRESS, Set.of(AWAITING_CONFIRMATION),
        AWAITING_CONFIRMATION, Set.of(COMPLETED),
        COMPLETED, Set.of(),
        CANCELLED, Set.of()
    );

    public boolean canTransitionTo(JobStatus target) {
        return ALLOWED_TRANSITIONS.getOrDefault(this, Set.of()).contains(target);
    }
}
```

Recommended reading: Refactoring Guru — State Pattern (understanding the full pattern clarifies when to upgrade to it); *Effective Java* (Joshua Bloch), Item 34 — use enums instead of int constants.

11. REST API Contract

Conventions: resource-based plural-noun URLs; HTTP verbs carry meaning (POST create, GET read, PATCH partial update for status transitions); versioned from day one under /api/v1; DTOs cross the wire, never JPA entities directly, keeping the database schema from leaking into the API contract.

Auth

Method	Endpoint	Auth	Purpose
POST	/api/v1/auth/register	none	Register as CLIENT or ARTISAN
POST	/api/v1/auth/login	none	Returns JWT
POST	/api/v1/auth/refresh	refresh token	Issue new access token

Artisan Profiles

Method	Endpoint	Auth	Purpose
GET	/api/v1/artisans	none (public)	Search/browse: trade, lat, lng, radiusKm, minRating
GET	/api/v1/artisans/{id}	none	Full profile: bio, rate, trades, rating, portfolio
PUT	/api/v1/artisans/me	ARTISAN	Update own profile
POST	/api/v1/artisans/me/portfolio	ARTISAN	Upload portfolio media
POST	/api/v1/artisans/me/qualifications	ARTISAN	Add a qualification

Jobs (core resource)

Method	Endpoint	Auth	Purpose
POST	/api/v1/jobs	CLIENT	Create job (origin: OPEN_POST or DIRECT_SEARCH)
GET	/api/v1/jobs/{id}	owner Client or relevant Artisan	Job detail
GET	/api/v1/jobs/{id}/matches	CLIENT (owner)	Ranked artisan matches for an OPEN_POST job
GET	/api/v1/jobs/me	CLIENT or ARTISAN	List own jobs
PATCH	/api/v1/jobs/{id}/status	varies by transition	Body: targetStatus, validated via JobStatus guard

Job Responses & Reviews/Payments

Method	Endpoint	Auth	Purpose
POST	/api/v1/jobs/{jobId}/responses	ARTISAN	Submit a proposal on an open job
GET	/api/v1/jobs/{jobId}/responses	CLIENT (owner)	View all proposals
PATCH	/api/v1/jobs/{jobId}/responses/{id}	CLIENT (owner)	Accept/reject a response

POST	/api/v1/jobs/{jobId}/review	CLIENT (owner)	Only if job status = COMPLETED
POST	/api/v1/jobs/{jobId}/payment	CLIENT (owner)	Initiates Paystack charge
GET	/api/v1/payments/{id}/verify	CLIENT (owner)	Webhook-independent verification

No generic PUT /jobs/{id}: status changes are deliberately routed only through the dedicated PATCH .../status endpoint so the state-machine guard clause is the sole path a status can change through.

Authorization is per-resource, not just per-role: role checks (e.g. hasRole('CLIENT')) confirm the user type but not resource ownership. Service-layer checks (e.g. authenticated user's ID equals the job's client_id) are required on top of role-based authorization to prevent one client from accessing another's data.

Recommended reading: Microsoft REST API Guidelines (github.com/microsoft/api-guidelines); Martin Fowler, “Richardson Maturity Model.”

12. Security Design

JWT Structure & Claims

The subject claim (sub) is the user's UUID, not their email, since a stable internal identifier should not change if contact details do. The role claim is embedded directly in the token so Spring Security can authorize a request without a database round-trip on every call — the core benefit of statelessness. The trade-off: a role change does not take effect until the existing token expires; acceptable for v1 since roles are fixed at registration.

```
{
  "sub": "user-uuid-here",
  "role": "ARTISAN",
  "iat": 1735900000,
  "exp": 1735903600
}
```

Two-Token Strategy

Token	Lifetime	Stored where	Purpose
Access token	15 minutes	Client memory/header	Sent on every API request
Refresh token	7 days	HttpOnly cookie	Used only to obtain a new access token

A single long-lived token would give an attacker a full week of access if stolen. Splitting into a short-lived access token plus a refresh token stored in an HttpOnly cookie (unreadable by JavaScript) limits the damage window and closes off the most common theft vector (XSS). This mirrors standard production practice.

Password Storage

BCrypt via Spring Security's BCryptPasswordEncoder (work factor 10–12). Never SHA-256 or similar fast hashes for passwords — speed is the wrong property for password hashing, since it is exactly what makes brute-forcing cheap. BCrypt is deliberately slow and applies a per-password salt automatically.

Authorization Layering

Role-level checks (e.g. @PreAuthorize("hasRole('CLIENT')")) confirm user type only. Resource-level ownership — confirming a specific job belongs to or is assigned to the requesting user — is enforced inside the service layer, not the annotation, since ownership is data-dependent rather than role-dependent.

```
@PreAuthorize("hasRole('CLIENT')")
@PostMapping("/api/v1/jobs")
public ResponseEntity<JobDto> createJob(...) { ... }

// resource-level check lives in the service:
public JobDto getJob(UUID jobId, UUID requestingUserId) {
    Job job = jobRepository.findById(jobId).orElseThrow(...);
    if (!job.belongsToOrIsAssignedTo(requestingUserId)) {
        throw new AccessDeniedException("Not authorized to view this job");
```

```
        }
        ...
    }
```

Recommended reading: OWASP Top 10 (A01 Broken Access Control, A02 Cryptographic Failures); Spring Security JWT resource server reference documentation.

13. Matching / Ranking Algorithm

Step 1: Hard Filters

Applied as SQL WHERE conditions, not scored: trade matches the job's required trade; distance (via PostGIS ST_DWithin) is within the client's specified or default radius; artisan account is active.

Step 2: Weighted Scoring

```
score = (0.5 x normalizedRating)
      + (0.3 x proximityScore)
      + (0.2 x portfolioCompletenessScore)
```

Component	Calculation	Rationale
normalizedRating	Bayesian-adjusted avg_rating / 5.0	Highest weight: strongest trust signal
proximityScore	1 - (distanceKm / radiusKm)	Meaningful but shouldn't override a much better-rated artisan
portfolioCompletenessScore	min(1.0, (photos + videos x2 + quals x3) / 10)	Lowest weight: self-reported, not outcome-verified

The Cold-Start Problem

A brand-new artisan with zero reviews should not score 0 on rating, or every new artisan would be buried permanently. A Bayesian average solves this by blending the artisan's own average with an assumed neutral prior, weighted by a fixed number of “phantom” reviews:

```
normalizedRating = (avgRating x numReviews + priorMean x priorWeight)
                   / (numReviews + priorWeight)

// e.g. priorMean = 3.5, priorWeight = 5
```

A new artisan starts near the middle of the pack, and the score becomes increasingly their own genuine average as real reviews accumulate, rather than starting at zero or receiving an unearned perfect score.

Why This Is Scoring, Not a Recommendation Engine

Every component above is deterministic math, traceable by hand for any given artisan — no model training, no weights that drift over time. This keeps the feature inside backend engineering scope rather than drifting into a separate machine-learning discipline.

Recommended reading: Evan Miller, “How Not To Sort By Average Rating”; Wikipedia — “Bayesian average.”

14. Exception Handling Strategy

Custom Exception Hierarchy

```
public abstract class ApplicationException extends RuntimeException {
    public ApplicationException(String message) { super(message); }
}

public class ResourceNotFoundException extends ApplicationException { ... }         // 404
public class InvalidJobStateTransitionException extends ApplicationException { ... } // 409
public class AccessDeniedException extends ApplicationException { ... }             // 403
public class PaymentProcessingException extends ApplicationException { ... }        // 502
public class ValidationException extends ApplicationException { ... }               // 400
```

Distinct exception types — rather than one generic exception with an error-code field — let @ControllerAdvice route each to the correct HTTP status, and let service code throw a meaningful exception without needing to know or care what HTTP status eventually applies. Business logic throws meaningful exceptions; the web layer alone decides what HTTP status each one means.

Global Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(ex.getMessage(), "RESOURCE_NOT_FOUND"));
    }

    @ExceptionHandler(InvalidJobStateTransitionException.class)
    public ResponseEntity<ErrorResponse> handleInvalidTransition(InvalidJobStateTransitionException ex
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(new ErrorResponse(ex.getMessage(), "INVALID_STATE_TRANSITION"));
    }

    // ... one handler per exception type

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleUnexpected(Exception ex) {
        log.error("Unhandled exception", ex);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("An unexpected error occurred", "INTERNAL_ERROR"));
    }
}
```

Every error response follows a consistent shape: { message, errorCode, timestamp }. This lets any API consumer branch on errorCode rather than parsing English message text.

Security note: the catch-all handler logs the real exception server-side but returns only a generic message to the client. Leaking stack traces or internal exception details in API responses is a genuine information-disclosure vulnerability.

Recommended reading: Baeldung, “Error Handling for REST with Spring”; OWASP guidance on improper error handling (related to A02/A05 categories).

15. Testing Strategy

Testing follows the classic testing pyramid: many fast unit tests, fewer integration tests against real infrastructure, and a handful of end-to-end tests covering critical flows only. Unit tests are cheap to write and run in milliseconds, so they dominate by volume; integration tests spin up real infrastructure (slower, more brittle) so there are fewer; E2E tests exercise the whole system and are the slowest and most brittle, so only enough exist to prove the critical path works end-to-end.

Unit Tests (no Spring context, no database)

- JobStatus.canTransitionTo() — every valid and invalid transition pair
- The matching/scoring formula — given known inputs, assert the exact expected score
- DTO ↔ entity mapping logic
- Pure business logic in services, with repositories mocked via Mockito

```
@Test
void shouldRejectTransitionFromCompletedToInProgress() {
    assertFalse(JobStatus.COMPLETED.canTransitionTo(JobStatus.IN_PROGRESS));
}

@Test
void shouldCalculateBayesianRatingCorrectly() {
    double result = ratingCalculator.normalizedRating(4.8, 2, 3.5, 5);
    assertEquals(3.76, result, 0.01); // (4.8*2 + 3.5*5) / 7
}
```

Integration Tests (real Spring context + real Postgres via Testcontainers)

Repository queries — especially the PostGIS distance query, since spatial queries behave differently against a real index than “in your head”; the full JobService.createJob() flow; security checks such as a CLIENT hitting an ARTISAN-only endpoint correctly receiving 403. H2 is deliberately not used here since it lacks PostGIS support — testing against H2 would mean not testing the query that matters most in this project. Testcontainers provides a real, disposable Postgres+PostGIS container per test run.

End-to-End Tests (sparingly — 3–5 tests total for v1)

1. Register → login → create job (open post) → get matches
2. Full happy path: post → assign → accept → progress → confirm → pay → review
3. Auth failure paths: expired token, wrong role

Principle: Test Behavior, Not Implementation

```
// Bad -- tests HOW, breaks on any internal refactor
verify(jobRepository, times(1)).save(any());

// Good -- tests WHAT, survives refactors
Job saved = jobService.createJob(request);
assertEquals(JobStatus.OPEN, saved.getStatus());
```

Recommended reading: Martin Fowler, “TestPyramid”; Testcontainers documentation (Spring Boot + Postgres module); Vladimir Khorikov, *Unit Testing Principles, Practices, and Patterns*.

16. Payment Integration Flow (Paystack)

The core design challenge is not calling the payment API — it is handling the case where money moves but the server does not hear about it in time (network blip, restart, delayed webhook). Getting this wrong risks double-charging a client or leaving a job stuck showing unpaid when it was not.

Flow

1. Client triggers payment via POST /api/v1/jobs/{jobId}/payment
2. Backend calls Paystack's Initialize Transaction endpoint, receives an authorization_url and a reference (generated by the backend, not Paystack)
3. Backend saves a Payment row immediately with status PENDING and that reference, before the client completes payment
4. Client is redirected to Paystack's hosted checkout page — card details are never touched directly, which matters for both security and PCI-DSS compliance
5. Client completes payment on Paystack's page
6. Two independent confirmations: a webhook call from Paystack to the backend, and a Verify Transaction call made by the frontend after redirect, as a fallback if the webhook is delayed or missed

```
@PostMapping("/api/v1/webhooks/paystack")
public ResponseEntity<Void> handleWebhook(@RequestBody String payload,
                                          @RequestHeader("x-paystack-signature") String signature) {
    if (!paystackSignatureValidator.isValid(payload, signature)) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
    }
    paymentService.processWebhookEvent(payload);
    return ResponseEntity.ok().build();
}
```

Why signature validation is non-negotiable: without it, anyone who knows the webhook URL could POST a fake “payment successful” event and have a job marked paid for free. Paystack signs every webhook with the account's secret key (HMAC SHA512); that signature is verified before the payload is trusted at all.

Idempotency

Both the webhook and the verify-endpoint fallback may fire for the same payment. processWebhookEvent must be idempotent — calling it twice with the same reference has the same effect as calling it once.

```
public void processWebhookEvent(String reference, PaymentStatus newStatus) {
    Payment payment = paymentRepository.findByReference(reference).orElseThrow(...);
    if (payment.getStatus() == PaymentStatus.SUCCESS) {
        return; // already processed -- do not double-trigger job completion
    }
    payment.setStatus(newStatus);
    paymentRepository.save(payment);
    if (newStatus == PaymentStatus.SUCCESS) {
        jobService.transitionStatus(payment.getJobId(), JobStatus.COMPLETED);
    }
```

```
    }
```

This guard clause is the entire idempotency strategy — simple, but it is the detail that separates “works in the demo” from “works when Paystack retries a webhook because the server was briefly down,” which happens in production more often than expected.

Recommended reading: Paystack API docs — Transactions and Webhooks guides; Stripe's “Idempotent Requests” documentation — widely considered the clearest industry explanation of the concept, and directly transferable despite using a different provider.

17. Implementation Order

Bridges the design and implementation phases. Features are built as vertical slices — entity, repository, service, controller, and tests for one feature at a time — rather than horizontally by layer, so something working and provably correct exists at every step rather than only once nearly everything has been built.

1. Project skeleton + User auth

Register, login, JWT. Every other feature needs an authenticated user; also the simplest feature that still touches Spring Boot config, Spring Security, JPA, and exception handling together.

2. ArtisanProfile (CRUD only)

Straightforward CRUD tied to User — reinforces a 1-to-1 JPA relationship before harder ones are introduced. No search or location yet.

3. Trade + ArtisanTrade

Isolated step to learn the many-to-many join table pattern on its own.

4. Basic Job creation + the state machine

Direct requests only first (the simpler origin), no matching yet. This is where JobStatus and the guard-clause logic are implemented and unit tested.

5. Open posts + JobResponse

Layered in once direct requests work end-to-end; naturally harder since multiple artisans can respond.

6. Location + PostGIS + matching/ranking algorithm

Deliberately placed after jobs work functionally without it — an enhancement to a working flow, not a prerequisite.

7. Portfolio media + qualifications

Needed to give the portfolio-completeness part of the scoring formula real data to test against.

8. Reviews

Straightforward once Job reliably reaches COMPLETED.

9. Paystack payment integration

Deliberately second-to-last — the most operationally complex piece (webhooks, idempotency, an external dependency), sequenced once the rest of the domain is a known quantity.

10. Notifications (in-app)

Lowest priority; every earlier feature already works without it.

Why This Order

The sequence moves from simple/isolated to complex/interdependent, and from core domain to external integrations — a deliberate risk-reduction strategy. Payment integration is the single most likely step to

consume unpredictable time (webhook debugging, sandbox quirks), so it is sequenced once everything else is already a known quantity, rather than while Spring Boot fundamentals are still being learned.

Working Process for the Build Phase

For each step: a brief concept explanation, code written with the underlying logic explained in simple terms, the user explains back what they understand the code to be doing, review and correction, then tests written together before moving to the next slice. This combines code-reading practice with code-writing familiarity.